Trabalho Prático

Onde o Modelo Escuta?

Embeddings + Attention para Reconhecimento de Comandos de Voz

25 de setembro de 2026

Objetivo

Neste trabalho você irá treinar um classificador de **comandos de voz reais** e investigar duas partes internas do modelo: os **embeddings** aprendidos e os **pesos de Attention**. O objetivo não é implementar Attention do zero. Usaremos a implementação pronta do TensorFlow/Keras para concentrar o esforço em uma pergunta mais importante: **como a representação aprendida e a atenção participam da decisão final?**

1 Cenário

O Problema

Imagine um pequeno robô controlado por voz. Ele precisa ouvir uma palavra curta e decidir uma ação. Se alguém disser `left`, ele deve virar à esquerda; se disser `right`, deve virar à direita; `go` significa andar; `stop`, parar. O computador não recebe a palavra escrita: recebe apenas uma onda sonora.

O fluxo que iremos estudar é:

$$\boxed{\text{Áudio}} \rightarrow \boxed{\text{Espectrograma}} \rightarrow \boxed{\text{Embedding}} \rightarrow \boxed{\text{Attention}} \rightarrow \boxed{\text{Classificação}}$$

A pergunta central do trabalho será:

O que o modelo aprendeu a representar, onde ele concentra atenção e como isso se relaciona com a decisão?

2 Dataset real: Mini Speech Commands

Usaremos o **Mini Speech Commands**, disponibilizado no tutorial oficial de reconhecimento de áudio do TensorFlow. Ele contém milhares de gravações reais de comandos curtos, organizadas em oito classes:

`yes, no, up, down, left, right, stop, go`

O tutorial oficial está disponível em:

https://www.tensorflow.org/tutorials/audio/simple_audio

O conjunto pode ser baixado diretamente pelo código abaixo:

```python
import pathlib
import tensorflow as tf

DATASET_PATH = 'data/mini_speech_commands'
data_dir = pathlib.Path(DATASET_PATH)

if not data_dir.exists():
    tf.keras.utils.get_file(
        'mini_speech_commands.zip',
        origin='http://storage.googleapis.com/download.tensorflow.org/
           data/mini_speech_commands.zip',
        extract=True,
        cache_dir='.', cache_subdir='data')
```

Intuição

O dataset já é conhecido, pequeno o suficiente para uma atividade didática e usado no próprio tutorial do TensorFlow. Portanto, o objetivo deste trabalho não é "caçar um dataset" nem construir toda a infraestrutura de áudio. O foco é experimentar e interpretar.

3 Etapa 1 — Enxergando o áudio

Escolha exemplos de pelo menos quatro comandos diferentes. Para cada exemplo, observe a forma de onda e transforme o áudio em um **espectrograma**.

O TensorFlow já fornece as operações necessárias:

```python
def get_spectrogram(waveform):
    waveform = waveform[:16000]
    zero_padding = tf.zeros([16000] - tf.shape(waveform), dtype=tf.
       float32)
    waveform = tf.cast(waveform, dtype=tf.float32)
    equal_length = tf.concat([waveform, zero_padding], 0)

    spectrogram = tf.signal.stft(
        equal_length,
        frame_length=255,
        frame_step=128)

    return tf.abs(spectrogram)
```

Experimento

Produza o **Gráfico 1: forma de onda** e o **Gráfico 2: espectrograma**. Compare dois áudios da mesma palavra, preferencialmente pronunciados por pessoas diferentes. Explique por que os gráficos não são idênticos mesmo representando o mesmo comando.

4 Etapa 2 — Construindo o embedding

Agora transformaremos o espectrograma em uma sequência de vetores. Não é necessário construir um extrator sofisticado. Uma pequena rede é suficiente para o experimento.

Um exemplo de bloco é:

```python
from tensorflow.keras import layers

x = layers.Conv1D(64, 3, padding='same', activation='relu')(entrada)
```

```
x = layers.MaxPooling1D(2)(x)

embedding = layers.Dense(
    64,
    activation='relu',
    name='embedding'
)(x)
```

Se a entrada da Attention tiver formato

$$(B, T, D),$$

então B é o tamanho do lote, T é o número de posições temporais e D é a dimensão de cada embedding.

Intuição

Pense em cada posição temporal como um "token de áudio". Em NLP tínhamos uma sequência de palavras ou subwords. Aqui temos uma sequência de pequenas regiões do áudio. Cada região é representada por um vetor aprendido.

Experimento: os embeddings aprenderam alguma organização?

Depois de treinar o modelo, extraia embeddings de aproximadamente 200 a 300 exemplos do conjunto de teste. Para obter **um vetor por áudio**, você pode calcular a média das posições temporais.

Depois use PCA:

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
embeddings_2d = pca.fit_transform(embeddings)
```

Experimento

Produza o **Gráfico 3: PCA dos embeddings**. Cada ponto deve representar um áudio e a classe deve ser identificável no gráfico. Observe especialmente dois pares, por exemplo `left/right` e `go/stop`. Comandos da mesma classe tendem a ocupar regiões semelhantes? Há classes misturadas?

5 Etapa 3 — Attention pronta do Keras

Não implemente Q, K, V ou o softmax manualmente. Neste trabalho usaremos a camada pronta:

```
attention_layer = layers.MultiHeadAttention(
    num_heads=2,
    key_dim=32,
    name='attention'
)

attention_output = attention_layer(
    query=embedding,
    value=embedding,
    key=embedding
)
```

Por dentro, a ideia continua sendo a estudada em sala:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Atenção

A biblioteca está fazendo a implementação, mas isso não elimina o conceito. O trabalho de vocês é interpretar o comportamento da camada: quais regiões temporais recebem mais peso? As cabeças fazem exatamente a mesma coisa? O comportamento muda quando o modelo erra?

Para recuperar os pesos de atenção, a camada pode retornar também os *attention scores*:

```
attention_output, attention_scores = attention_layer(
    query=embedding,
    value=embedding,
    key=embedding,
    return_attention_scores=True
)
```

Os scores podem então ser visualizados com `matplotlib` usando `imshow()`.

6 Etapa 4 — Tomando a decisão

Depois da Attention, reduza a dimensão temporal e faça a classificação:

```
x = layers.GlobalAveragePooling1D()(attention_output)
x = layers.Dropout(0.2)(x)
saida = layers.Dense(8, activation='softmax')(x)
```

Uma saída possível será semelhante a:

```
Arquivo: exemplo.wav

yes      0.01
no       0.02
up       0.01
down     0.00
left     0.91   <-- decisao
right    0.03
stop     0.01
go       0.01
```

A Decisão

Separe **5 previsões corretas e 5 previsões incorretas**. Não mostre somente a classe prevista: mostre também as probabilidades produzidas pelo softmax. Um erro com 51% de confiança é diferente de um erro com 99%.

7 Etapa 5 — Onde o modelo prestou atenção?

Esta é a principal análise do trabalho.

Escolha:

- um áudio classificado corretamente;

- um áudio classificado incorretamente.

Para cada um, visualize os pesos de Attention como um heatmap.

```python
import matplotlib.pyplot as plt

# Exemplo: primeira amostra, primeira cabeca
A = attention_scores[0, 0].numpy()

plt.imshow(A, aspect='auto')
plt.xlabel('Key: tempo')
plt.ylabel('Query: tempo')
plt.title('Mapa de Attention - cabeca 1')
plt.colorbar()
plt.show()
```

Experimento

Produza o **Gráfico 4: Attention de um acerto** e o **Gráfico 5: Attention de um erro**. Responda: (1) a atenção ficou aproximadamente uniforme ou existem regiões de maior peso? (2) as duas cabeças apresentam o mesmo padrão? (3) o exemplo errado apresenta alguma diferença visual interessante em relação ao exemplo correto?

Atenção

Cuidado com a interpretação: um mapa de Attention é uma evidência sobre os pesos usados pelo mecanismo, mas não prova sozinho que determinada região foi a "causa" da decisão. Descreva o que o gráfico mostra antes de tentar explicar por quê.

8 Etapa 6 — Attention realmente ajudou?

Agora faremos a comparação mais importante do trabalho. Treine duas versões do modelo, mantendo o restante o mais parecido possível.

Modelo A — sem Attention

Espectrograma → Embedding → GlobalAveragePooling → Dense

Modelo B — com Attention

Espectrograma → Embedding → MultiHeadAttention → GlobalAveragePooling → Dense

Experimento

Produza o **Gráfico 6: accuracy de validação por época**, contendo as curvas dos dois modelos. Monte também uma pequena tabela com a accuracy final de teste de cada modelo.

Não existe resposta que deva ser forçada. Se o modelo com Attention tiver resultado melhor, explique a evidência. Se ficar praticamente igual ou pior, isso também é um resultado válido.

Pergunta obrigatória: a Attention ajudou neste experimento? Que evidências sustentam sua conclusão?

9 Etapa 7 — Onde o modelo erra?

Para o modelo com Attention, produza uma matriz de confusão usando o conjunto de teste.
Você pode usar:

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_true, y_pred)
ConfusionMatrixDisplay(cm, display_labels=nomes_classes).plot()
```

Este será o **Gráfico 7: matriz de confusão**.
Depois responda:

1) Quais comandos o modelo mais confunde?
2) Essas classes aparecem próximas ou misturadas no PCA dos embeddings?
3) Observe um erro envolvendo essas classes. O mapa de Attention apresenta alguma característica diferente?

10 A história que o grupo precisa contar

O trabalho não deve ser uma coleção de gráficos desconectados. A apresentação precisa seguir esta sequência:

Áudio → Espectrograma → Embedding → Attention → Probabilidades → Decisão

Para pelo menos um áudio, o grupo deverá explicar cada etapa.

Intuição

A pergunta não é apenas "o modelo acertou?". Queremos descobrir: **como o áudio foi representado, onde a Attention concentrou peso e qual decisão apareceu no final?**

11 Critérios de avaliação

Critério	Nota
Pipeline completo funcionando e código organizado	2,0
Análise e visualização dos embeddings	2,0
Visualização e interpretação da Attention	2,0
Comparação experimental sem Attention vs. com Attention	2,0
Apresentação, saída do código e interpretação dos erros	2,0
Total	**10,0**

O Problema

Pergunta final para a apresentação: olhando conjuntamente para o PCA dos embeddings, os mapas de Attention, a matriz de confusão e as probabilidades produzidas pelo modelo, o que vocês aprenderam sobre a forma como a rede transforma um sinal de áudio em uma decisão?